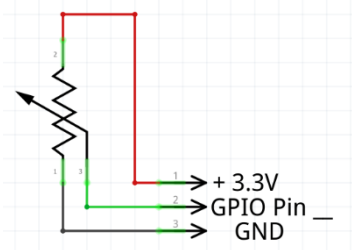


Vaja 1 – posamična ADC pretvorba z Nucleo

1. **Cilj naloge:** S pomočjo programskega okolja **STM32CubeIDE** in HAL knjižnicami sprogramirajte ARM mikroprocesor tako, da bo izvedel posamične ADC pretvorbe z izbranim potenciometrom.
2. **Postopek inicializacije periferije.**
 - a) Zaženite **STM32CubeIDE** in ustvarite nov STM32 projekt (pod zavihkom *information Center*). V zavihku *Board selector* s pomočjo filtrov, Type in MCU/MPU Series Izberite ustrezno razvojno ploščo (v našem primeru Nucleo-L476RG), kliknite *Next*, projekt poimenujte **vaja1_ADC_pretvorba_Sk_X** in kliknite *Next*, v oknu za knjižnice uzberite *Add necessary library files as reference ..* ter gumb *Finish* (na možnosti opcije za inicializacije periferije izberite *Yes*, izbrana naj bo tudi opcija perspektive za STM32CubeMX).
 - b) V *Pinout* pogledu si oglejte prerazporeditev PIN-ov mikroprocesorja za vhodne in izhodne enote. Zelena LED je priključena na PA6 pin. Kaj je pa priključeno na pin PC13? Moder gumb.
 - c) V *Analog* razdelku levo od sheme mikroprocesorja ugotovite, koliko ADC pretvornikov ima vaša razvojna ploščica? 3
 - d) Izbrani ADC pretvornik(i) ima(jo) oznako s trikotnikom. Kaj to pomeni?
To pomeni, da je pretvornik delno izklopljen - nekateri pini so uporabljeni za kaj drugega
Kaj morate storiti, da razrešite to omejitev? Opišite in **jo odpravite**.
Zadržimo miško nad trikotnikom, da nam program pove kje je konflikt in ga odpravimo. V mojem primeru sem moral izklopiti PA6 pin iz gpio na disabled in moral sem izklopiti USART2
 - e) Razširite (kliknite) razdelek *ADC3*. Koliko je vseh vhodnih kanalov (seštejte oznake INxx) ?
4
 - f) Izberite (obkljukajte) poljuben prosti analogni kanal oz.vhod, ki ga boste porabili za branje vrednosti iz potenciometra. Izbrati morate *Single-ended* princip zajemanja analogne vrednosti. Na zaslonu se vam mora usterzno pobarvati izbrani pin v zeleno barvo. Kaj se izpiše poleg pina? ADC1_IN1 Kateri pin je to? PC0
 - g) Ustrezno priključite priključke potenciometra na Nucleo ploščico (slika)!
 - h) V kolikor ste prej vgrajeno LED diodo deaktivirali, jo ponovno aktivirajte na ustreznem (dig.) izhodu/pinu, (glej *vaja0a*).
 - i) Pod *Configuration ADC* enote gumb v *Parameter settings* izberite ločljivost pretvorbe na **8-bitno**, torej je območje vrednosti od 0 ÷ 255. Preglejte, kakšne so še ostale možne ločljivosti pretvorbe in območja vrednosti?
 - a. 6 bit, od 0 do 63,
 - b. 10 bit, od 0 do 1023,
 - c. 12 bit, od 0 do 4096.
 - j) Sedaj generirajte kodo tako, da enostavno kliknete ikono *Save* in po potrebi še enkrat potrdimo generiranje kode, potrdimo tudi izbiro perspektive v C/C++.



Vaja 1 – posamična ADC pretvorba z Nucleo

3. Programiranje v IDE:

- V skrajno levem oknu *Project Explorer* poiščemo **main.c** datoteko pod *Core* → *Src* → *main.c* (dvokliknite na datoteko, odpre se tekstovni urejevalnik za main.c).
- V neskočni zanki programa `While (1)` pod *User code begin 3* zapišite ukaz za branje analogne vrednosti in ukaz za 100 ms zakasnitev. Branje ADC pretovrnika nastavimo z ukazom:

```
HAL_ADC_Start(&ADC_HANDLE_TYPE);
```

Ime tega paramtera boste našli v vrstici pod *Private variables* ----. Del zgornje kode v rdečem torej zamenjajte z imenom te ročice (`ADC_HandleTypeDef XXXX`).

- Naslednji ukaz `HAL_ADC_PollForConversion` služi za inicializacijo »bazena« podatkov za potrebe pretvorbe ter čas same pretvorbe, ki naj bo 5 ms. Vrača nam tudi uspešnost pretvorbe, zato zapišemo vejitev:

```
if (HAL_ADC_PollForConversion(&XXXX, čas_pretvorbe_v_ms) == HAL_OK)
{
}
}
```

- Znotraj vejitve (ko je pogoj izpolnjen) vpišemo ukaz za zajem vrednosti:

```
adcValue = HAL_ADC_GetValue(&XXXX);
```

Izmerjena vrednost se bo shranila v spremenljivko *adcValue*.

- V *User code begin 0* deklariramo našo ADC spremenljivko kot pozitivno 32-bitno:

```
uint32_t adcValue;
```

- Pretvorbo moramo tudi ustaviti. Za vejitvijo napišemo:

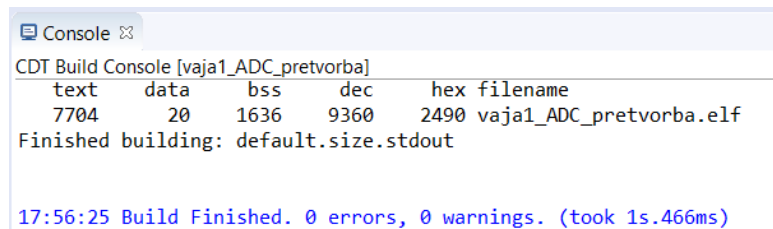
```
HAL_ADC_Stop(&XXXX);
```

- Ste vpisali ukaz za 100 ms delay? Če še niste, ga dodajte (izven if vejitve!):

```
HAL_?????(?);
```

4. Naložitev kode (Run)

- Kodo preverite s tipko **Build** (ikona za klavirce - *Debug*). Ko je preverjanje končano lahko preverimo, če smo med pisanjem kode naredili kakšno napako sintakse, sicer se pod kodo v oknu *Console* izpiše 0 errors:



```
CDT Build Console [vaja1_ADC_pretvorba]
text    data    bss     dec     hex filename
7704    20      1636    9360    2490 vaja1_ADC_pretvorba.elf
Finished building: default.size.stdout

17:56:25 Build Finished. 0 errors, 0 warnings. (took 1s.466ms)
```

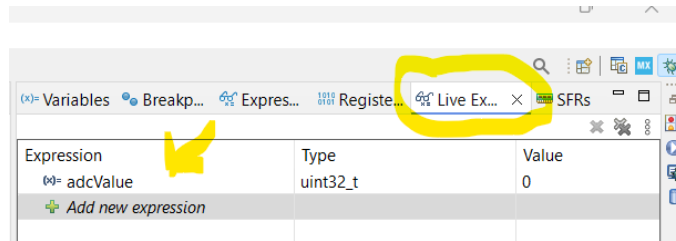
- Priklopite STM32 Nucleo na vaš računalnik preko USB kabla.
- S tipko **Run** (zelena ikona za play – *Run as STM...*) prenesete program na STM32 Nucleo.
- V oknu *Edit Configuration* kliknemo OK. Nekaj sekund bo na ploščici STM izmenično utripala zelena in rdeča LED, ko je program naložen, sveti LED rdeče.

5. Branje vrednosti spremenljivke znotraj IDE (debug mode)

- Kliknemo puščico poleg Debug ikone in izberemo iz spustnega seznama *Debug Configurations* ..
- V zavihu *Debugger* omogočimo *Serial Wire Viewer (SWV)*, frekvenco jedra pustimo privzeto.
- Še ni preverimo, da je *Enable live expressions* omogočen.
- Kliknemo na gumb *Apply*, nato še *Debug*. Sedaj se okolje spremeni v razhroščevalni (debug) način.

Vaja 1 – posamična ADC pretvorba z Nucleo

- e) V iskalniku (prazno belo polje desno zgoraj) vpišemo **Live expressions (Debug)**, izberemo najdeno opcijo ter v tabelo pod *Expression* vpišemo ime naše spremenljivke **adcValue** in pritisnemo tabulator. V kolikor je zapis spremenljivke pravilen, se samodejno izpolnita naslednji polji pod *Type* in *Value*.



- a) Sedaj lahko kliknemo na ikono *Resume* (lahko pritisnemo tudi tipko F8). V tabeli *Live Expressions* bi morali videti trenutno vrednost spremenljivke oz. pretvorbe v realnem času (če zavrtite potenciometer). **Delovanje posnamite s telefonom.**
- f) Za ustavitev branja kliknemo gumb *Terminate* (ali CTRL + F2) in se tako vrnemo v pogledu programiranja.

6. Branje vrednosti s programom STM Studio (če branje ne deluje v debug mode!)

- b) Zaženite program STM Studio.
- c) Kliknite na *Import variables from executable*. Kliknemo ikono ... in poiščemo našo **.elf** datoteko (nahaja se v mapi vašega projekta).
- d) V nastali tabeli izberite vaše spremenljivke `adcValue` in kliknite *Import*. Kliknite *Close*.
- e) Na levi strani (okno *Display Variables*) najprej označite `adcValue` in nato desni klik na `adcValue` ter jo pošljite na *VarViewer1*. V *Viewer Settings* namesto *Bar Graph* izberemo *Table*. Kliknite ikono za RUN (ne pozabite resetirati STM!).
- f) Sedaj lahko spreminjate pozicijo potenciometra in opazujete vrednost spremenljivke `adcValue`. Delovanje posnamite s telefonom.

7. **Vaš projekt** (datoteko *main.c* [15%], slikovni izrezek *Pinout* mikroporcesorja iz CubeMX [15%], kratek videoposnetek delovanja [15%], fotografija izbranega potenciometra na razširitveni ploščici [10%]) **naložite v Github** kot nov *Repository* (public!) z imenom **Vaja1-ADC-single-conversion-Nucleo** [5%]. V *readme* datoteki zapišite vse **odgovore** [30%] na vprašanja ter **komentar** [10%] na delovanje. **Oceno** pridobite iz Github dokumentacije!

8. Dodatna video pomoč z razlago: [Video Tutorial 3](#).

9. Pinout NUCLEO-L476RG:

